Deliverable 6: Data Strategy

What data this needs, how it was simulated, what real data would take

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

Data Strategy

What I actually needed

I wasn't going to get real Nucor data. That's stated outright in the assignment, and honestly it's the more interesting constraint of the whole exercise. So before writing a line of generation code, I asked myself what the *minimum realistic signal* was to actually prove the idea: that a model can see a bearing failure coming before the alarm does. Not a generic "sensor dataset," something with a real precursor pattern baked in, or the whole exercise is theater.

I decided on five signals per roll stand, sampled every minute: vibration, bearing temperature, motor current, line speed, and coolant pressure. I didn't pick these because they were easy to fake. I picked them because between them they cover three different failure signatures (mechanical, thermal, electrical), one operational-response signal, and one that moves in the *opposite* direction from the rest. I've made the mistake before of loading a model up with features that are all correlated and teach it nothing new, and wanted to avoid doing that again here.

Column	Signal type	Why it's there
`vibration_rms_mm_s`	Mechanical	The standard bearing-wear precursor in real predictive-maintenance literature
`bearing_temp_c`	Thermal	Friction from wear generates heat, slower and noisier than vibration, so it's a differently-shaped signal, not a redundant one
`motor_current_a`	Electrical	Motor works harder against a degrading bearing, ties the mechanical fault to something a plant's electrical/PLC system would already log
`line_speed_mpm`	Operational response	Operators throttle back speed as a stand starts acting up, tests whether the model can read human responses to a developing fault, not just raw physics
`coolant_pressure_psi`	Inverse signal	Drops as the seal degrades, while everything else rises, forces the model to learn an actual pattern instead of thresholding one direction

How I built it

First attempt was lazy: normal noise, then a sudden step-change right before the failure timestamp. It looked fake the moment I plotted it: flat, then a cliff. No real machine dies like a light switch.

So I rebuilt the failure lead-up as a non-linear ramp. Starting somewhere between 6 and 48 hours before the failure, each signal drifts toward a "failing" value on a `progress^2.2` curve, slow at first, accelerating near the end, because that's closer to how bearing wear actually behaves. I also widen the noise as the fault develops rather than just shifting the mean, since degrading equipment gets noisier, not just offset. When I finally plotted coolant pressure trending down into a real failure event instead of just jittering around a mean, that was the moment I trusted the dataset enough to build a model on top of it.

The generator (`data/generate_synthetic_data.py`) is seeded, so it's fully reproducible. I didn't check the 25MB output into the repo, I checked in the script that produces it deterministically. Output: 6 stands times 45 days at 1-minute resolution (about 389K rows), with roughly 22% of stand-days ending in an injected failure, paired with a `failure_events.csv` ground-truth label file.

Update once the live tier existed: the 45 day window used to start on a hardcoded date, January 1st. That was fine for training but became a real problem once the dashboard could filter by Today, Past week, or Last month, since the live feed only knows about "now" onward and the historical window was frozen months in the past, leaving a huge silent gap between the two. The generator now anchors its window to end yesterday, computed off the real clock whenever it runs, instead of a fixed date. Same seed, same signal sequences, only the calendar labels move, so nothing about the actual data or the trained model changed, it just stays current instead of drifting further into the past every day this sits unrun. Full story, including a retraining mistake I almost made over this exact change, in `docs/ai-partnership-log.md`.

Where this is honestly weak

This is my best guess at realistic magnitudes and failure dynamics, not something measured off a real mill. The five signals, the 22% failure rate, the 6 to 48 hour degradation window: all reasonable assumptions, none of them verified against actual Nucor equipment. I'd rather say that plainly here than pretend the numbers are more grounded than they are.

What I'd ask Nucor's data team for, if this were real

- **Historian access** (likely OSIsoft PI or equivalent) for the actual tag names mapping to vibration, temperature, current, speed, and pressure on a specific roll stand, plus their real sampling rates. I assumed 1-minute resolution, but plant historians often log slower or event-triggered, which changes the whole modeling approach.
- **A real failure log**: actual unplanned downtime events with timestamps and root cause, ideally from a CMMS (maintenance management system), so the model trains against ground truth instead of my simulated labels.
- **Reliability engineering sign-off on the signal list itself**: whether these five are actually the leading indicators for this specific failure mode at this specific mill, or whether there's a sixth signal (oil analysis, acoustic emission, something else entirely) that matters more in practice than anything I guessed at.
- **Data quality expectations**: sensor dropout rates, calibration drift, how stale a hearth-side sensor reading is allowed to get before it's untrustworthy. This is the same category of concern the JD calls out for MDM validation. Anomaly detection is only as good as the data quality guarantees underneath it, and I'd want that conversation with the data team before trusting this in production, not after.